

Stored XSS into Anchor href Attribute with Double Quotes HTML-Encoded

Lab Overview

This lab demonstrates a **Stored Cross-Site Scripting (XSS)** vulnerability involving an HTML anchor (`<a>`) element.

The application stores attacker-controlled input and later places it into an `href` attribute. Although double quotes are HTML-encoded, the attacker can exploit the URL scheme and attribute context to execute JavaScript when the stored link is activated.

Objective

Exploit the stored XSS vulnerability by injecting a malicious value into an anchor's `href` attribute.

Vulnerability

Vulnerability Type: Stored Cross-Site Scripting (XSS)

Context: HTML `href` attribute

The application stores user-controlled input and later renders it as an anchor destination without restricting dangerous URL schemes.

Methodology

1. Open the vulnerable lab.
2. Locate a form or functionality that allows user input to be stored.
3. Submit a controlled test value.
4. Inspect where the stored value appears.
5. Determine that the value is inserted into an anchor's `href` attribute.
6. Test the handling of quotation marks and HTML encoding.
7. Use a JavaScript URL scheme as the attribute value.
8. Submit the malicious value.
9. Return to the page containing the stored link.
10. Activate the link and observe JavaScript execution.

Exploitation

A representative payload is:

```
javascript:alert(document.domain)
```

When the application places this value into the anchor's `href`, clicking the resulting link can cause the browser to interpret the URL using the JavaScript scheme.

The vulnerability is therefore related not only to HTML encoding but also to unsafe handling of URL schemes.

Result

The malicious value is stored by the application and later rendered as an anchor destination. Activating the link results in JavaScript execution.

This confirms a stored XSS vulnerability.

Impact

Stored XSS is particularly significant because the malicious input persists in the application and can affect users who later view the affected content.

Potential consequences include:

- Arbitrary JavaScript execution
- Unauthorized actions in the victim's session
- Manipulation of application content
- Theft of information accessible to JavaScript
- Phishing attacks
- Persistent attacks against other users

Remediation

Developers should:

- Validate URLs before placing them into `href` attributes.
- Allow only approved URL schemes such as `https:` where appropriate.
- Explicitly reject dangerous schemes such as `javascript:.`
- Apply context-appropriate output encoding.
- Sanitize user-generated HTML when HTML input is actually required.

- Use Content Security Policy as an additional defense.

OWASP Reference

OWASP Top 10: A03 – Injection

Related vulnerability: Stored Cross-Site Scripting (XSS)

Conclusion

This lab demonstrates that HTML-encoding alone does not necessarily make an `href` attribute safe. Applications must also validate the URL scheme and ensure that user-controlled values cannot introduce executable protocols.

GitHub Commit Message

Add stored XSS href attribute report